

Project Report on
VISION-BASED HUMAN FOLLOWING ROBOT
USING RASPBERRY PI

Submitted by

Swayam Das (2341004102)

Anubhab Patra (2341016261)

Smruti Sourav Mishra (2341013151)

Saadiya Farheen (2341006003)

S. Jayant Kumar (2341011103)

Veetesh Sinha (2341019574)

Mayan Shankar (2341019399)

B. Tech. (CSE (IOT)) 6th Semester (Section–23412O3)



CENTRE FOR INTERNET OF THINGS
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
Institute of Technical Education and Research

SIKSHA 'O' ANUSANDHAN
DEEMED TO BE UNIVERSITY

Bhubaneswar, Odisha, India.
(June, 2026)

Abstract

The “Vision Based Human Following Robot” is a smart robotic vehicle that can detect and follow a person using a camera and computer vision techniques. The project is built using a Raspberry Pi 4 Model B, Pi Camera v1.3, TB6612FNG motor driver, and a robotic car kit. The robot captures live video through the camera and processes the frames using OpenCV and TensorFlow Lite. A lightweight object detection model is used to identify a human in front of the robot. Based on the position of the person in the video frame, the robot moves left, right, forward, or stops to follow the target smoothly.

Power is supplied using rechargeable batteries and regulated through two buck converters to provide stable voltage to the Raspberry Pi and motor drivers. The software part of the project uses Python libraries such as OpenCV, Picamera2, and TensorFlow Lite Runtime for image processing and real-time control.

The main aim of this project is to develop a low-cost and efficient human-following robot that can work in real time. This system can be useful in applications such as smart assistance, surveillance, luggage carrying systems, and service robots. The project also helps in understanding computer vision, embedded systems, artificial intelligence, and robotics through practical implementation.

Contents

Abstract	i
Contents	ii
Chapter 01: Introduction	1
1.1. Introduction	1
1.2. Background	1
1.3. Project Objectives	1
1.4. Scope	2
1.5. Project Management	2
1.6. Overview and Benefits	3
1.7. Organization of the Report	3
Chapter 02: Background Review & Survey	4
2.1. Related Works	4
2.2. Existing Technologies	4
2.3. Camera Based Human Detection	4
2.4. Human Detection Methods	5
2.5. Advantages of vision-based robots	5
2.6. Limitations of existing systems	5
Chapter 03: Theoretical Aspects	6
3.1. Introduction	6
3.2. Computer Vision	6
3.3. Object Detection	6
3.4. OpenCV	6
3.5. TensorFlow Lite	6
3.6. Raspberry Pi	7
3.7. Motor Control	7
3.8. Power Supply System	7
3.9. Project Layout	7
3.10. Summary	8
Chapter 04: Hardware Requirements	9
4.1. Introduction	9
4.2. Raspberry Pi 4B	9
4.3. Pi Camera v1.3	10
4.4. TB6612FNG Motor Driver Module	10

4.5 DC Motors & Car Chasis.....	11
4.6. Buck Converter.....	12
4.7. Rechargeable Battery.....	12
4.8. Power Supply Connections.....	12
4.9. Block Diagram.....	13
4.10. Working of the system.....	13
4.11. Circuit Diagram.....	13
4.12. Components Required.....	14
4.13. Summary.....	14
Chapter 05: Software Requirements.....	15
5.1. Introduction.....	15
5.2. Operating System.....	15
5.3. Python Programming Language.....	15
5.4. OpenCV.....	16
5.5. TensorFlow Lite Runtime.....	16
5.6. Pi Camera-2 Library	16
5.7. GPIO Library.....	17
5.8. Software Tools Used	17
5.9. System Requirements.....	17
5.10 Software Logic.....	18
5.11 Logic and Flowchart.....	18
5.12 Flowchart Description.....	19
5.13 Summary.....	20
Chapter 06: Project Development and Testing Aspects.....	21
6.1. Hardware setup and interfacing.....	21
6.2. Software Development and model deployment.....	21
6.3. System Integration and Logic flow.....	21
6.4. Testing methodologies.....	22
Chapter 07: Conclusion Future Scope.....	23
7.1. Results.....	23
7.2. Limitations	23
7.3. Further enhancement and future scope.....	24
References.....	2

Appendix

01.....26

<u>A01.1. Standalone deployment Scripts</u>	26
<u>A01.2. Main Code</u>	27
<u>A01.3. Libraries</u>	32

Appendix 02.....33

<u>A02.1. Project Proposal Form</u>	33
<u>A02.2. Project Management</u>	33
<u>A02.3. Bill of Material</u>	34

Appendix

03.....35

<u>A03.1. Raspberry Pi 4B</u>	35
<u>A03.2. TB6612FNG Motor Driver</u>	35
<u>A03.3. Raspberry Pi Camera v1.3</u>	35
<u>A03.4. Power Supply System</u>	36

Chapter 01: Introduction

1.1. Introduction

Robots are becoming an important part of modern technology. They are used in industries, healthcare, security systems, smart homes, and many other areas. A human following robot is a type of robot that can detect and follow a person automatically. In this project, a vision-based human following robot is developed using computer vision and image processing techniques.

The robot uses a camera to capture live video and identify a person in front of it. The Raspberry Pi processes the video frames and controls the movement of the robot. The robot moves forward, left, or right depending on the position of the person.

This project combines robotics, embedded systems, and artificial intelligence to create a smart and low-cost human following system.

1.2. Background

Earlier human following systems mainly used infrared sensors, Bluetooth modules, or RF transmitters for tracking a person. These methods had limited range and accuracy. With the growth of computer vision and machine learning, camera-based tracking systems have become more effective and reliable.

The development of low-cost devices such as the Raspberry Pi 4 Model B has made it easier to build smart robotic systems. Libraries such as OpenCV and TensorFlow Lite help in real-time image processing and object detection. These technologies allow robots to detect and follow humans without requiring any special transmitter or wearable device.

This project uses these modern technologies to create a simple and efficient human following robot.

1.3. Project Objectives

The main objectives of this project are:

- To design and develop a vision-based human following robot.
- To detect and track a person using a Pi Camera.
- To process live video using OpenCV and TensorFlow Lite Runtime.
- To control the movement of the robot using Raspberry Pi.
- To create a low-cost and real-time robotic system.
- To understand the practical use of computer vision and robotics.

1.4. Scope

The scope of this project includes the development of a vision-based human following robot that can detect and follow a person in real time using a camera and computer vision techniques. The system uses a Raspberry Pi, Pi Camera, OpenCV, and TensorFlow Lite to identify a person and control the movement of the robot based on the person's position in the camera frame.

The project can be further enhanced in the future by adding:

- Face recognition features
- Voice control
- GPS tracking
- Mobile application control
- Multi-person tracking
- Outdoor navigation support

The system can be used in applications such as smart assistance, surveillance, hospitals, shopping malls, warehouses, and service robot.

1.5. Project Management

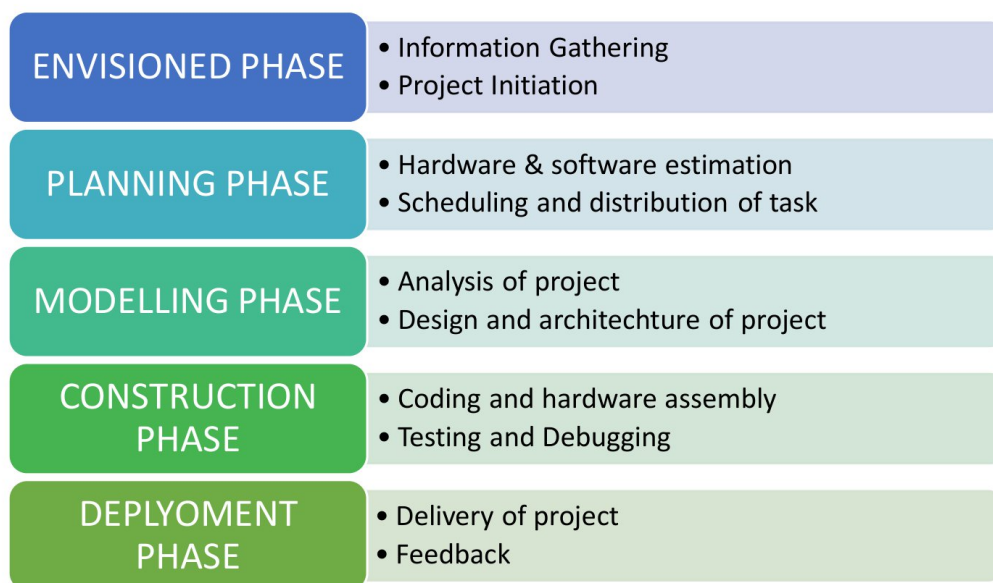


Figure 1.1. Model of phases in project management.

The project was completed by dividing the work into hardware setup, software development, testing, and documentation. Proper planning and execution were carried out to ensure the successful completion of the project.

The hardware phase involved assembling the robotic car chassis, connecting the Raspberry Pi, Pi Camera, TB6612FNG motor drivers, DC motors, batteries, and buck converters. The software phase included installing the required libraries such as OpenCV, Picamera2, and TensorFlow Lite Runtime, as well as developing the Python program for human detection, centroid-based tracking, and robot movement.

Testing was performed regularly to verify human detection, motor control, camera performance, and overall robot movement. Finally, the project results were analyzed and documented in the report.

1.6. Overview and Benefits

The vision-based human following robot uses a camera to capture live video and detect a human using machine learning techniques. The Raspberry Pi processes the data and controls the motors to follow the person.

Benefits of this project include:

- Real-time human tracking
- Low-cost implementation
- Easy hardware setup
- Practical use of artificial intelligence
- Improved understanding of robotics and computer vision
- Useful for automation and smart assistance systems

The project also provides hands-on experience in programming, image processing, and embedded systems.

1.7. Organization of the Report

The report is organised into different chapters to explain the project clearly and systematically.

- **Chapter 1: Introduction**
This chapter gives an introduction to the project, objectives, scope, and overview of the system.
- **Chapter 2: Background Review & Survey**
This chapter discusses previous works, related technologies, and existing human following systems.
- **Chapter 3: Theoretical Aspects**
This chapter explains the theoretical concepts used in the project such as computer vision, and object detection.
- **Chapter 4: Hardware Requirements**
This chapter describes the hardware components used in the project including the Raspberry Pi 4 Model B, Pi Camera, motor driver, batteries, and robotic car kit.
- **Chapter 5: Software Requirements**
This chapter explains the software tools and libraries used for the project such as Python, OpenCV, picamera2, and TensorFlowLite.
- **Chapter 6: Project Development & Testing Aspects**
This chapter explains the implementation process, system integration, testing methods, results, and observations of the project.

Chapter 02: Background Review & Survey

2.

2.1. Related Works

Many researchers have developed human following robots using different technologies and methods. Earlier systems mainly used infrared sensors, Bluetooth modules, and RF transmitters to track a person. These methods were simple but had limited range and lower accuracy.

With the development of computer vision and artificial intelligence, camera-based human following systems became more popular. These systems use image processing techniques to detect and track humans in real time. Researchers commonly use devices such as the Raspberry Pi 4 Model B along with machine learning models for low-cost robotic applications.

Several projects also use OpenCV and TensorFlow Lite for object detection and tracking. These technologies improve the performance, accuracy, and real-time working of human following robots.

The related works studied during this project helped in understanding the advantages and limitations of existing systems and guided the development of the proposed robot.

2.2 Existing Technologies

Different technologies are used in human following robots depending on the application and design requirements. Some commonly used technologies are:

- Computer vision for human detection
- Machine learning models for object recognition
- Motor drivers for movement control
- Embedded systems such as Raspberry Pi

These technologies work together to help the robot detect a person, process information, and move automatically.

2.3 Camera-Based Human Detection

Camera-based systems are more advanced compared to traditional sensor-based systems.

In this method, the camera captures live video frames and image processing techniques are used to detect humans.

OpenCV is used for image processing, while TensorFlow Lite models help in object detection. The robot identifies the position of the person and moves accordingly. Camera-based systems provide better flexibility and accuracy for real-time applications.

2.4 Human Detection Methods

Human detection is an important part of vision-based robotic systems. Different computer vision and machine learning techniques are used to identify and track people in images and video streams.

In this project, human detection is performed using a Pi Camera, OpenCV, and TensorFlow Lite. The camera captures live video frames, which are processed by the Raspberry Pi. A TensorFlow Lite object detection model is used to identify a person in the video frame. Based on the detected position of the person, the robot adjusts its movement and follows the target in real time.

2.5 Advantages of Vision-Based Robots

Vision-based robots provide several advantages such as:

- Better human tracking accuracy
- Real-time object detection
- Reduced dependency on external devices
- Easy system expansion
- Low-cost implementation using Raspberry Pi

These advantages make vision-based robots suitable for modern automation systems.

2.6 Limitations of Existing Systems

Some existing human-following robots have the following limitations:

- High implementation cost
- Slow processing speed in real-time applications
- Reduced performance in low-light environments
- Difficulty in accurately tracking a person in complex backgrounds
- Complex hardware and software integration

These limitations motivated the development of a simpler, cost-effective, and efficient vision-based human following robot system.

Chapter 03: Theoretical Aspects

3.

3.1 Introduction

Theoretical aspects explain the basic concepts and technologies used in the development of the vision-based human following robot. This project combines computer vision, embedded systems, machine learning, and robotics to create an automated human tracking system.

The robot detects a person using a camera, processes the image using Raspberry Pi, and controls the movement of motors based on the position of the detected person.

3.2 Computer Vision

Computer vision is a technology that allows computers and robots to understand and process images and videos. In this project, computer vision is used to detect and track a person from live camera footage.

The Pi Camera captures continuous video frames, and image processing techniques are applied to identify the human target. Based on the detected position, the robot changes its direction and movement.

3.3 Object Detection

Object detection is a technique used to identify and locate objects in an image or video. In this project, object detection is used to recognize humans from live video frames.

TensorFlow Lite models are used for lightweight and faster detection on Raspberry Pi. The system checks each video frame and detects the presence of a person in real time.

3.4 OpenCV

OpenCV is an open-source computer vision library used for image and video processing. It provides functions for capturing frames, processing images, drawing boundaries, and object tracking.

In this project, OpenCV is used to:

- Capture live video frames
- Process camera images
- Display detection results
- Assist in human tracking

OpenCV helps improve the speed and efficiency of the system.

3.5 TensorFlow Lite

TensorFlow Lite is a lightweight machine learning framework designed for embedded systems and mobile devices. It is used for running object detection models efficiently on Raspberry Pi.

The TensorFlow Lite model detects humans from camera input and provides the position of the detected object. This information is used to control robot movement.

3.6 Raspberry Pi

Raspberry Pi is a small single-board computer used for embedded and robotics applications. It acts as the main controller of the robot.

The Raspberry Pi:

- Processes camera input
- Runs the detection program
- Controls motor movement
- Reads sensor data

It provides enough processing power for real-time human detection and robot control.

3.7 Motor Control

The robot uses DC motors for movement. A motor driver module is used to control the motors because the Raspberry Pi cannot directly supply enough current to the motors.

The motor driver receives control signals from the Raspberry Pi and controls the direction and speed of the motors. Based on human detection, the robot moves forward, left, right, or stops.

3.8 Power Supply System

The Raspberry Pi is powered using a power bank, while the motors and sensors are powered using rechargeable batteries. Separate power supplies are used to maintain stable performance and prevent voltage drops during motor operation.

This setup improves system reliability and smooth robot movement.

3.9 Project Layout

The project layout shows the overall structure and connection of the components used in the vision-based human following robot. The system consists of hardware and software modules that work together for human detection and robot movement.

The Pi Camera captures live video and sends it to the Raspberry Pi for processing. OpenCV and TensorFlow Lite are used to detect and track a human from the video frames. Based on the position of the detected person, the Raspberry Pi sends control signals to the motor driver, which controls the movement of the DC motors.

The Raspberry Pi is powered using a power bank, while the motors are powered using rechargeable batteries for stable operation. This combination of hardware and software enables the robot to follow a person in real time.

Main Components of the Project Layout

- Raspberry Pi 4B
- Pi Camera v1.3

- Motor Driver Module
- DC Motors and Car Chassis
- Power Bank
- Rechargeable Batteries
- OpenCV and TensorFlow Lite Software

Working Flow

- The Pi Camera captures live video continuously.
- The Raspberry Pi processes the video frames.
- Human detection is performed using TensorFlow Lite and OpenCV.
- The position of the detected person is identified.
- The Raspberry Pi controls the robot movement based on the person's position.
- The robot moves forward, left, right, or stops accordingly.

3.11 Summary

This chapter explained the theoretical concepts used in the project, including computer vision, object detection, OpenCV, TensorFlow Lite, Raspberry Pi, and motor control. These concepts form the foundation of the vision-based human following robot system.

Chapter 04: Hardware Requirements

4.

4.1 Introduction

Hardware components play an important role in the development of the vision-based human following robot. These components are responsible for image capture, data processing, motor control, robot movement, and power management. The Raspberry Pi, Pi Camera, motor drivers, DC motors, batteries, and buck converters work together to enable real-time human detection and following. Proper selection and integration of hardware components ensure reliable and efficient operation of the system.

4.2 Raspberry Pi 4B

The Raspberry Pi 4B is the main controller of the robot. It processes camera input, runs the human detection program, and controls the movement of the robot.

Features

- Processor: Quad-core Cortex-A72 64-bit SoC @ 1.5 GHz
- RAM: 2GB, 4GB, or 8GB LPDDR4 options
- Operating Voltage: 5V via USB-C
- Connectivity: Gigabit Ethernet, Wi-Fi 802.11 b/g/n/ac, Bluetooth 5.0
- USB Ports: 2 × USB 3.0 and 2 × USB 2.0
- GPIO Pins: 40 pins for peripheral interfacing
- Video Output: 2 × micro HDMI ports supporting 4K display
- Storage: microSD card support for operating system and data storage

Uses in Project

- Human detection processing
- Running OpenCV and TensorFlow Lite
- Motor control
- Processing camera input

4.2.1 Raspberry Pi 4B GPIO Pin Configuration

The GPIO pins of the Raspberry Pi are used to connect the motor driver module for controlling the movement of the robot.

GPIO Pin	Connection
GND	Common Ground
GPIO 17	Motor Driver IN1
GPIO 18	Motor Driver IN2
GPIO 27	Motor Driver IN3

These GPIO pins are used to control the direction and movement of the DC motors through the motor driver module.

4.3 Pi Camera v1.3

The Pi Camera is used to capture live video frames for human detection and tracking.

Features

- Resolution: 5 Megapixel
- Video Recording: Supports 1080p video
- Interface: CSI camera interface
- Fixed focus camera module
- Compact and lightweight design
- Direct Raspberry Pi compatibility

Uses in Project

- Capturing live video
- Providing input for object detection

4.3.1 Pi Camera v1.3 Pin Configuration

The Pi Camera is connected directly to the Raspberry Pi using the CSI camera connector.

Camera Pin	Connection
CSI Ribbon Cable	Raspberry Pi CSI Port

Camera Interface Details

- Interface Type: CSI (Camera Serial Interface)
- Direct connection to Raspberry Pi motherboard
- Used for live video capture

4.4TB6612FNG Motor Driver Module

The TB6612FNG motor driver module is used to control the movement of the DC motors. It receives control signals from the Raspberry Pi and drives the motors efficiently.

Features

- Dual H-Bridge motor driver
- Operating Voltage: 2.5V to 13.5V
- Low power consumption

- High motor driving efficiency
- Supports forward and reverse motor control
- Compact and lightweight design

Uses in Project

- Controlling DC motor movement
- Forward and reverse motion
- Left and right turning
- Receiving control signals from Raspberry Pi

4.4.1 TB6612FNG Pin Configuration

Two TB6612FNG motor driver modules are used to control the four DC motors of the robot. The motor drivers receive control signals from the Raspberry Pi and drive the motors accordingly.

TB6612FNG Pin	Raspberry Pi Connection
PWMA	GPIO 12
AIN1	GPIO 17
AIN2	GPIO 18
PWMB	GPIO 13
BIN1	GPIO 27
BIN2	GPIO 22
STBY	3.3V
GND	GND
VM	Battery Supply
VCC	3.3V

The output pins A01, A02, B01, and B02 of the motor drivers are connected to the four DC motors. Together, the two motor drivers control the movement of all four wheels of the robot.

4.5 DC Motors and Car Chassis

DC motors provide movement to the robot, while the chassis supports all hardware components.

Features

- High torque geared DC motors
- Suitable for robotic vehicle applications
- Durable chassis structure
- Easy mounting support for components

Uses in Project

- Robot movement
- Hardware support and stability

4.5.1 DC Motor Connections

The DC motors are connected to the output terminals of the TB6612FNG motor driver. The motor driver receives control signals from the Raspberry Pi and controls the movement of the motor.

Motor	TB6612FNG Connection
Left Motors	A01 and A02
Right Motors	B01 and B02

The TB6612FNG motor driver controls the direction and movement of the motors based on the signals received from the Raspberry Pi.

4.6 Buck Converter

A buck converter is a DC-DC voltage regulator used to step down the battery voltage to the required operating voltage of the system components. In this project, two buck converters are used to provide stable and efficient power distribution.

Features

- High power conversion efficiency
- Stable voltage output
- Compact size
- Adjustable output voltage

Uses in Project

- Supplying regulated voltage to the Raspberry Pi
- Supplying regulated voltage to the motor drivers
- Protecting components from voltage fluctuations

4.7 Rechargeable Batteries

Rechargeable lithium-ion batteries are used as the main power source for the robot. The batteries provide power to the Raspberry Pi, motor drivers, and DC motors through the buck converters.

Features

- Lithium-ion rechargeable batteries
- Output Voltage: 3.7V each
- Rechargeable and reusable
- High energy density
- Suitable for robotic applications

Uses in Project

- Providing power to the Raspberry Pi
- Providing power to the TB6612FNG motor drivers
- Powering the DC motors
- Supporting continuous robot operation

4.8 Power Supply Connections

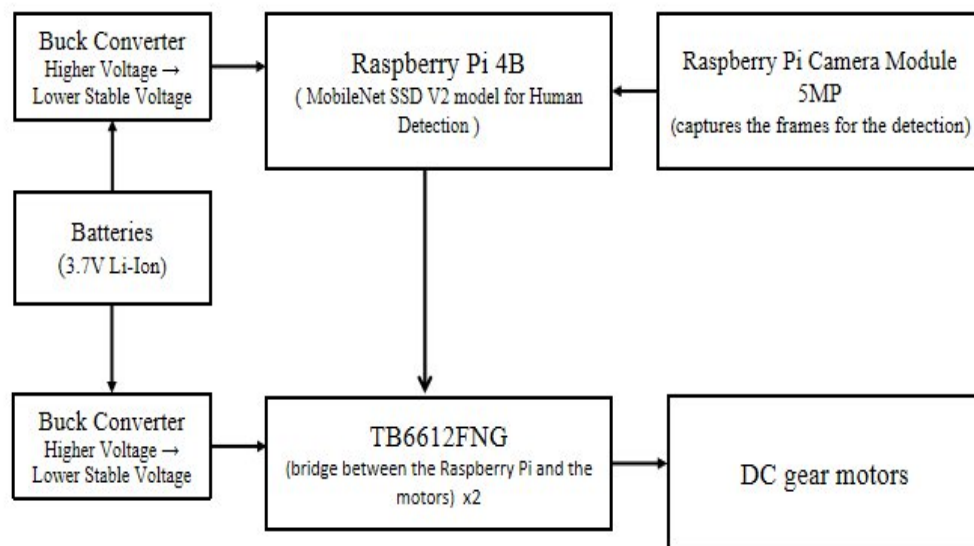
The robot uses rechargeable batteries as the main power source. Two buck converters are used to provide stable voltage to different components of the system.

Component	Power Source
Raspberry Pi 4B	Battery Pack through Buck Converter 1
TB6612FNG Motor Drivers	Battery Pack through Buck Converter 2
DC Motors	Motor Driver Output

The use of buck converters ensures that the required voltage is supplied safely and efficiently to the Raspberry Pi and motor drivers, resulting in stable system operation.

4.9 Block Diagram

The block diagram shows the overall structure and connection of the hardware components used in the vision-based human following robot system.



4.10. Working of the System

The working of the system begins with the Pi Camera continuously capturing live video frames. These video frames are sent to the Raspberry Pi for processing.

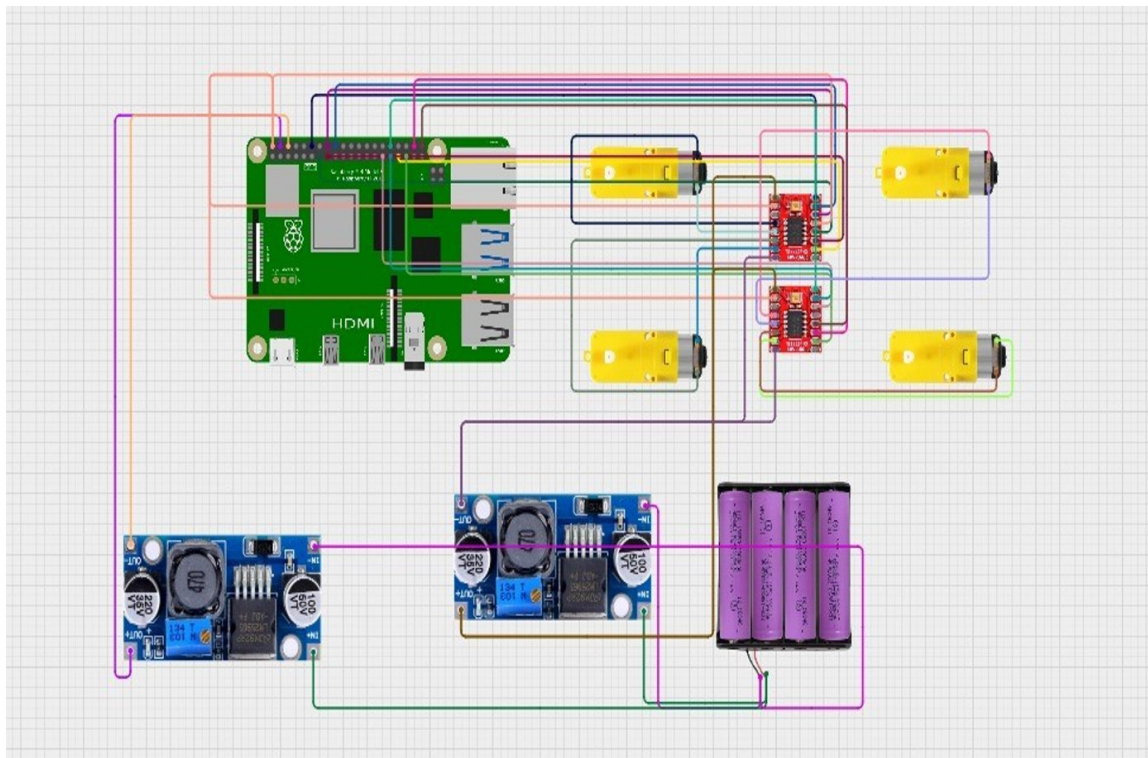
The Raspberry Pi uses OpenCV and TensorFlow Lite to detect a human from the captured video. Once a person is detected, a bounding box is generated around the detected person, and the centroid of the bounding box is calculated. The centroid position is then compared with the center of the camera frame.

Based on the centroid position, the Raspberry Pi sends control signals to the two TB6612FNG motor drivers. The motor drivers control the four DC motors, allowing the robot to move forward, turn left, turn right, or stop in order to follow the detected person.

The system is powered by rechargeable batteries, and two buck converters are used to provide stable voltage to the Raspberry Pi and motor drivers. This enables the robot to perform real-time human detection and following efficiently.

4.11 Circuit Diagram

The circuit diagram shows the hardware connections among the Raspberry Pi 4B, Pi Camera, TB6612FNG motor driver, and DC motors. These connections allow the Raspberry Pi to process camera input and control the movement of the robot based on human detection.



4.12 Components Required

The following table shows the hardware components required for developing the vision-based human following robot.

Sl. No	Component Name	Quantity	Purpose
1	Raspberry Pi 4B	1	Main processing unit
2	Pi Camera v1.3	1	Capturing live video
3	TB6612FNG Motor Driver Module	2	Controlling DC motors
4	DC Motors	4	Robot movement
5	Robot Car Chassis	1	Supporting hardware components
6	Wheels	4	Robot movement
7	Buck Converter	2	Voltage regulation
8	Rechargeable Batteries	4	Power supply for motors
9	Jumper Wires	As Required	Hardware connections
10	Battery Holder	1	Holding batteries
11	Switch	1	Power control
12	Mounting Accessories	As Required	Component mounting

4.13 Summary

This chapter described the hardware components, pin configurations, block diagram, working process, circuit connections, and power supply arrangement of the vision-based human following robot. It explained the role of the Raspberry Pi, Pi Camera, TB6612FNG motor drivers, DC motors, batteries, and buck converters in the system. These hardware components work together to perform real-time human detection, centroid-based tracking, and robot movement efficiently.

Chapter 05: Software Requirements

5.1 Introduction

Software plays an important role in the working of the vision-based human following robot. The software tools and libraries are used for image processing, human detection, motor control, and communication between hardware components. The project is mainly developed using Python programming on the Raspberry Pi operating system.

5.2 Operating System

The Raspberry Pi uses Raspberry Pi OS for running the project programs and required libraries.

Features

- Linux-based operating system
- Supports Python programming
- Compatible with Raspberry Pi hardware
- Supports camera and GPIO interfacing

Uses in Project

- Running the robot control program
- Managing hardware and software communication

5.3 Python Programming Language

Python is the main programming language used for developing the project.

Features

- Simple and easy to understand
- Supports image processing libraries
- Large number of built-in modules
- Suitable for robotics and automation projects

Uses in Project

- Writing control programs
- Sensor interfacing
- Human detection processing
- Motor control

5.4 OpenCV

OpenCV is an open-source computer vision library used for image and video processing.

Features

- Real-time image processing
- Video frame capturing
- Object tracking support
- Easy integration with Python

Uses in Project

- Capturing camera frames
- Processing live video
- Assisting in human detection

5.5 TensorFlow Lite Runtime

TensorFlow Lite Runtime is used for running lightweight machine learning models on the Raspberry Pi.

Features

- Lightweight framework
- Faster object detection
- Optimized for embedded systems
- Supports real-time processing

Uses in Project

- Human detection
- Running object detection models

5.6 Picamera2 Library

Picamera2 is used for controlling the Pi Camera and capturing video frames.

Features

- Direct camera access
- Supports image and video capture
- Compatible with Raspberry Pi OS

Uses in Project

- Accessing Pi Camera
- Capturing live video feed

5.7 GPIO Library

GPIO libraries are used for controlling motors and interfacing sensors with the Raspberry Pi GPIO pins.

Features

- GPIO pin control
- Input and output support
- Easy hardware interfacing
- Real-time control of connected devices

Uses in Project

- Motor control through the TB6612FNG motor driver
- Sending control signals for robot movement
- Interfacing hardware components with the Raspberry Pi

5.8 Software Tools Used

Software/Library	Purpose
Raspberry Pi OS	Operating system
Python	Programming language
OpenCV	Image processing
TensorFlow Lite Runtime	Human detection
Picamera2	Camera interfacing
GPIO Library	Motor and sensor control

5.9 System Requirements

Requirement	Specification
Processor	Raspberry Pi 4B
RAM	Minimum 2GB
Storage	MicroSD Card
Programming Language	Python
Operating System	Raspberry Pi OS

5.10 Software Logic

The software logic controls the overall operation of the robot system. The Pi Camera continuously captures live video frames and sends them to the Raspberry Pi for processing.

The Raspberry Pi uses OpenCV and TensorFlow Lite to detect a human from the video frames. After detecting a person, the system determines the position of the person within the camera frame.

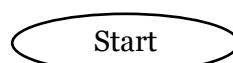
- If the person is in the center of the frame, the robot moves forward.
- If the person is on the left side of the frame, the robot turns left.
- If the person is on the right side of the frame, the robot turns right.
- If no person is detected, the robot stops.

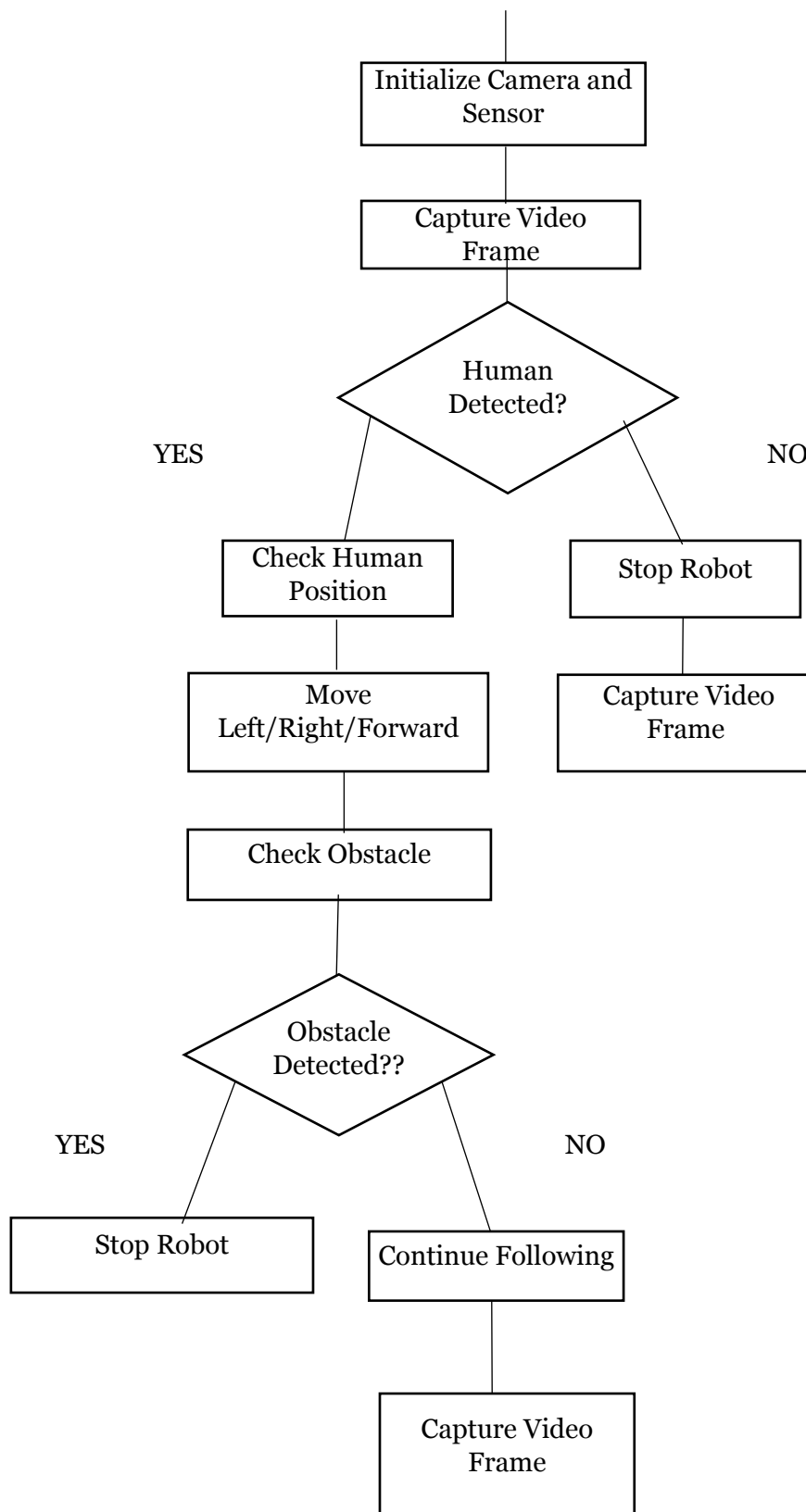
The Raspberry Pi continuously processes incoming video frames and updates the robot's movement in real time, enabling the robot to follow the detected person smoothly.

5.11 Logic and Flowchart

The flowchart represents the working sequence of the software system used in the robot.

(P.T.O.)





5.12 Flowchart Description

The system starts by initializing the Pi Camera and motor driver. The Pi Camera continuously captures live video frames and sends them to the Raspberry Pi for processing.

The Raspberry Pi uses OpenCV and TensorFlow Lite to detect a human in the captured video frames. When a person is detected, the system determines the position of the person within the camera frame and controls the robot's movement accordingly.

Based on the detected position, the robot moves forward, turns left, turns right, or stops. This process is repeated continuously, enabling the robot to follow the detected person in real time.

5.13 Summary

This chapter explained the software requirements, programming tools, software logic, and flowchart used in the vision-based human following robot project. The software system helps in image processing, human detection, obstacle detection, and robot movement control.

Chapter 06: Project Development & Testing Aspects

6.1 Hardware Setup & Interfacing

- **Core Computing Unit:** The system utilizes a Raspberry Pi as the central processing hub to handle real-time video capture, machine learning inference, and hardware control simultaneously.
- **Vision System Integration:** A Pi Camera module is interfaced using the picamera2 library, configured to capture video frames at a resolution of 320x240 pixels in the BGR888 format.
- **Motor Control Architecture:** The physical drive system is managed via two separate motor driver modules. GPIO pins are mapped to control the standby (STBY), directional (AIN/BIN), and speed control (PWM) pins for four distinct motor outputs, facilitating a comprehensive drive mechanism.
- **Pulse Width Modulation (PWM):** Software-based PWM is configured at a 1000Hz frequency to govern the base speed of the robot, ensuring smooth acceleration and consistent movement when a target is detected.

6.2 Software Development & Model Deployment

- **Pre-Trained Model Integration:** Instead of gathering data and training a custom network, the project leverages a highly optimized, pre-trained machine learning model: MobileNet SSD v2 (Single Shot MultiBox Detector) quantized and trained on the COCO dataset.
- **Edge Inference:** The TensorFlow Lite (.tflite) interpreter is utilized to load the model directly onto the Raspberry Pi. The system specifically filters the output to trigger solely when the person class is detected with a confidence score exceeding 45% (`SCORE_THRESHOLD = 0.45`).
- **Resource Optimization:** To prevent CPU bottlenecking on the edge device, inference is deliberately not run on every single video frame. A frame-skipping algorithm is implemented (`DETECTION_INTERVAL = 4`), evaluating the visual feed every fourth frame while maintaining real-time responsiveness.
- **Visual Feedback System:** OpenCV (cv2) is employed to dynamically draw bounding boxes around detected individuals in the live video feed, allowing for immediate visual debugging and system monitoring.

6.3 System Integration & Logic Flow

- The software and hardware are tied together via a continuous main loop. When the TFLite interpreter identifies a human within the bounding parameters, it triggers a boolean state change (`human_detected = True`), which subsequently commands the GPIO pins to drive all motors forward at the configured base speed.
- In the absence of a detected human, the system immediately drops the PWM duty cycles to zero (`stop_all()`), halting the robot to prevent erratic or unsafe movement.

6.4 Testing Methodologies

- **Component Testing:** Individual validation of the dual motor drivers to ensure correct polarity and wiring. PWM values were tuned to establish a stable BASE_SPEED (40%) that provides adequate momentum without compromising the camera's visual stability.
- **Vision & Model Testing:** The camera's focal length and lighting sensitivity were tested alongside the MobileNet SSD model to balance the threshold settings—ensuring the robot reacts to actual humans while minimizing false positives from background objects.
- **Integration Testing:** Rigorous testing of the main loop logic was conducted to verify that the transition between "HUMAN DETECTED -> MOVING" and "NO HUMAN -> STOP" occurs with minimal latency and that hardware cleanup executes safely upon system termination.

Chapter 07: Conclusion & Future Scope

7.1. Results

The implementation of the vision-based autonomous robotic system yielded successful real-time human detection and responsive physical actuation. The key outcomes of the project are as follows:

- **Reliable Edge Inference:** The system successfully deployed the quantized MobileNet SSD v2 model on the Raspberry Pi. By utilizing the TensorFlow Lite interpreter, the robot accurately identified humans within the camera frame, successfully filtering out background objects based on the configured confidence threshold (`SCORE_THRESHOLD = 0.45`).
- **Real-Time Actuation:** The integration between the machine learning pipeline and the GPIO hardware proved highly responsive. The system accurately toggled the motor driver states based on the `human_detected` boolean flag, initiating forward movement (`BASE_SPEED = 40%`) when a person was in the frame and executing an immediate, safe halt (`stop_all()`) when the frame was clear.
- **Resource Optimization:** The implementation of the frame-skipping algorithm (`DETECTION_INTERVAL = 4`) successfully balanced computational load. By processing every fourth frame rather than the entire video stream, the Raspberry Pi maintained a stable frame rate without overwhelming the CPU, while still providing smooth visual feedback and bounding box rendering via OpenCV.

7.2. Limitations

While the core objective of human detection and movement was achieved, the current prototype possesses several operational and architectural limitations:

- **Binary Movement Logic:** The current motor control logic is strictly linear (Forward or Stop). The robot lacks differential steering capabilities, meaning it cannot turn left or right to actively track a subject moving laterally across or out of the camera's field of view.
- **Generic Target Acquisition:** The MobileNet SSD model detects the general person class. If multiple humans enter the camera frame simultaneously, the robot cannot distinguish between them or lock onto a primary user; it will simply register that a human is present and move forward.
- **Lack of Depth Perception & Speed Scaling:** The system moves at a static `BASE_SPEED` regardless of how close or far the human is. There is no logic to estimate distance (e.g., using the bounding box area), meaning the robot will not slow down as it approaches the user, potentially leading to collisions.
- **Absence of Obstacle Avoidance:** Relying entirely on the camera for human detection means the robot is blind to environmental hazards. Without supplementary spatial sensors, the robot is susceptible to colliding with furniture or walls while attempting to move toward a detected human.
- **Lighting Dependency:** As a purely vision-based system relying on standard RGB video capture, the accuracy of the machine learning model heavily degrades in low-light environments or under severe glare.

7.3. Further Enhancement and Future Scope

To evolve the project from a functional prototype into a robust, autonomous following system, the following enhancements are proposed for future development:

- **Proportional-Integral-Derivative (PID) Steering:** Upgrading the motor control functions to calculate the center-point of the detected bounding box relative to the camera frame. By applying a PID controller to the left and right motor PWM signals independently, the robot could smoothly steer to keep the human centered in its vision.
- **Dynamic Distance Control:** Utilizing the total pixel area of the bounding box to estimate proximity. The system could dynamically adjust the PWM duty cycle—accelerating when the bounding box is small (subject is far) and decelerating to a stop as the bounding box grows larger (subject is close).
- **Hardware Acceleration Integration:** Activating the Edge TPU functionality already provisioned in the codebase (edgetpu=0) by installing a Google Coral USB Accelerator. This would offload neural network inference from the Pi's CPU, allowing for the use of more accurate, unquantized models or higher-resolution video feeds at 30+ frames per second.
- **Multi-Sensor Fusion for Safety:** Integrating ultrasonic sensors (e.g., HC-SR04) or a 2D LiDAR module to create an active obstacle-avoidance layer. This would override the forward movement command if a physical obstacle is detected in the robot's path, regardless of human detection.
- **Specific Subject Recognition:** Replacing or augmenting the MobileNet SSD object detection model with a facial recognition pipeline (such as OpenCV's FaceRecognizer or DeepFace). This would allow the robot to identify, lock onto, and follow a specific, authorized user in a crowded environment.

References

- Raspberry Pi Official Documentation

- [OpenCV Documentation](#)
- [TensorFlow Lite Documentation](#)
- [MobileNet SSD Object Detection Research Papers](#)
- [TB6612FNG Motor Driver Datasheet](#)
- [Research papers on Human Following Robots and Embedded AI Systems](#)
- [Python Documentation and GPIO Programming References](#)

Appendix 01

A01.1. Standalone Deployment Scripts

Step-1 : Give execute permission

```
chmod +x /home/pi/human_follow_robot/main.py
```

Step-2 : Create systemd Service

```
sudo nano /etc/systemd/system/humanfollow.service
```

```
[Unit]
```

```
Description=Human Following Robot
```

```
After=network.target
```

```
[Service]
```

```
Type=simple
```

```
User=pi
```

```
WorkingDirectory=/home/pi/method2_humandetect
```

```
ExecStartPre=/bin/sleep 10
```

```
ExecStart=/usr/bin/python3 /home/pi/method2_humandetect/cam_test.py
```

```
Restart=always
```

```
RestartSec=5
```

```
StandardOutput=journal
```

```
StandardError=journal
```

```
[Install]
```

```
WantedBy=multi-user.target
```

Step-3 : Reload system

```
sudo systemctl daemon-reload
```

Step-4 : Enable Auto Start

```
sudo systemctl enable humanfollow.service
```

Step-5 : Start immediately

```
sudo systemctl start humanfollow.service
```

Step-6 : Reboot Test

```
sudo reboot
```

Robot starts working automatically after 10-20 seconds.

A01.2. Main Code

Final rpi code: cam_test.py

```
import cv2
```

```
import numpy as np
```

```
from PIL import Image
```



```

import time
from picamera2 import Picamera2
import common as cm
import RPi.GPIO as GPIO

# =====
# MODEL CONFIG
# =====

MODEL_DIR = "."
MODEL_FILE = "mobilenet_ssd_v2_coco_quant_postprocess.tflite"
LABEL_FILE = "coco_labels.txt"

SCORE_THRESHOLD = 0.45
TOP_K = 3

# =====
# LOAD MODEL
# =====

interpreter, labels = cm.load_model(
    MODEL_DIR,
    MODEL_FILE,
    LABEL_FILE,
    edgetpu=0
)

# =====
# CAMERA SETUP
# =====

FRAME_W = 320
FRAME_H = 240

picam2 = Picamera2()

picam2.configure(
    picam2.create_preview_configuration(
        main={"size": (FRAME_W, FRAME_H),
            "format": "BGR888"}
    )
)

picam2.start()
time.sleep(1)

```

```

print("Camera Started")

# =====
# MOTOR DRIVER 1
# =====

PWMA1 = 18
AIN1_1 = 17
AIN2_1 = 27

BIN1_1 = 22
BIN2_1 = 5
PWMB1 = 19

STBY1 = 23

# =====
# MOTOR DRIVER 2
# =====

PWMA2 = 12
AIN1_2 = 6
AIN2_2 = 13
BIN1_2 = 20
BIN2_2 = 21
PWMB2 = 16

STBY2 = 24

GPIO.setmode(GPIO.BCM)

all_pins = [
    PWMA1, AIN1_1, AIN2_1,
    BIN1_1, BIN2_1, PWMB1, STBY1,

    PWMA2, AIN1_2, AIN2_2,
    BIN1_2, BIN2_2, PWMB2, STBY2
]

for pin in all_pins:
    GPIO.setup(pin, GPIO.OUT)

GPIO.output(STBY1, GPIO.HIGH)
GPIO.output(STBY2, GPIO.HIGH)

```

```

# =====
# PWM
# =====

pwmA1 = GPIO.PWM(PWMA1, 1000)
pwmB1 = GPIO.PWM(PWMB1, 1000)

pwmA2 = GPIO.PWM(PWMA2, 1000)
pwmB2 = GPIO.PWM(PWMB2, 1000)

pwmA1.start(0)
pwmB1.start(0)
pwmA2.start(0)
pwmB2.start(0)

BASE_SPEED = 40

# =====
# MOTOR FUNCTIONS
# =====

def move_forward(speed):

    # Driver 1 Motor A
    GPIO.output(AIN1_1, GPIO.HIGH)
    GPIO.output(AIN2_1, GPIO.LOW)

    # Driver 1 Motor B
    GPIO.output(BIN1_1, GPIO.HIGH)
    GPIO.output(BIN2_1, GPIO.LOW)

    # Driver 2 Motor A
    GPIO.output(AIN1_2, GPIO.HIGH)
    GPIO.output(AIN2_2, GPIO.LOW)

    # Driver 2 Motor B
    GPIO.output(BIN1_2, GPIO.HIGH)
    GPIO.output(BIN2_2, GPIO.LOW)

    pwmA1.ChangeDutyCycle(speed)
    pwmB1.ChangeDutyCycle(speed)

    pwmA2.ChangeDutyCycle(speed)
    pwmB2.ChangeDutyCycle(speed)

```

```

def stop_all():

    pwmA1.ChangeDutyCycle(0)
    pwmB1.ChangeDutyCycle(0)

    pwmA2.ChangeDutyCycle(0)
    pwmB2.ChangeDutyCycle(0)

# =====
# DETECTION SETTINGS
# =====

frame_count = 0
DETECTION_INTERVAL = 4

human_detected = False

# =====
# MAIN LOOP
# =====

try:

    while True:

        frame = picam2.capture_array()

        frame_count += 1

        if frame_count % DETECTION_INTERVAL == 0:

            rgb = cv2.cvtColor(
                frame,
                cv2.COLOR_BGR2RGB
            )

            pil_im = Image.fromarray(rgb)
            cm.set_input(interpreter, pil_im)

            interpreter.invoke()

            objs = cm.get_output(
                interpreter,
                score_threshold=SCORE_THRESHOLD,

```

```

        top_k=TOP_K
    )

    human_detected = False

    for obj in objs:

        if labels.get(obj.id, "") == 'person':

            human_detected = True

            h, w = frame.shape[:2]

            x0 = int(obj.bbox.xmin * w)
            y0 = int(obj.bbox.ymin * h)
            x1 = int(obj.bbox.xmax * w)
            y1 = int(obj.bbox.ymax * h)

            cv2.rectangle(
                frame,
                (x0, y0),
                (x1, y1),
                (0, 255, 0),
                2
            )

            break

# =====
# MOVEMENT
# =====

if human_detected:

    move_forward(BASE_SPEED)
    print("HUMAN DETECTED -> MOVING")

else:
    stop_all()
    print("NO HUMAN -> STOP")

cv2.imshow("Human Detection", frame)

if cv2.waitKey(1) & 0xFF == ord('q'):
    break

```

finally:

```
stop_all()
```

```
pwmA1.stop()
```

```
pwmB1.stop()
```

```
pwmA2.stop()
```

```
pwmB2.stop()
```

```
GPIO.cleanup()
```

```
picam2.stop()
```

```
cv2.destroyAllWindows()
```

A01.3. Libraries

RPi Python Libraries:

- cv2
- picamera2
- RPi.GPIO
- numpy
- PIL
- time
- common

Appendix 02

A02.1. Project Proposal Form

The project proposal form was prepared and duly signed from our Faculty-in-Charge **Dr. Shaktijeet Mohapatra and Dr. Bhanja Kishor Swain**. The same is attached at the last of this report.

A02.2. Project Management

Sl. No.	Component	Individual Contribution in %							Total
		Swayam Das	Anubhab Patra	Smruti Sourav Mishra	Veetesh Sinha	S. Jayant Kumar	Mayan Shankar	Saadiya Farheen	
1.	Planning		50			50			100%
2.	Background Research & Analysis	17	33	17		33			100%
3.	Hardware Design			30		35	35		100%
4.	Software Design	30	30			10	30		100%
5.	Testing	20	20		10	20	20	10	100%
6.	Final Assembling			25		25	50		100%
7.	Project Report Writing		10		35			55	100%
8.	Presentation				40			60	100%
9.	Logistics	10	15	25	25	25			100%
10.	Demonstration Video	20	20	20			20	20	100%

Table A2.2

A02.3. Bill of Material

#	Component	Unit Cost	Quantity	Total
1.	Raspberry Pi 4 Model B (4GB RAM)	11,449	1	11,449
2.	Pi Camerra v1.3	310	1	310
3.	RC Chassis Kit	649	1 set	649
4.	TB6612FNG Motor Driver	157	2	314
5.	LM2596 3A Buck Converter	68	2	136
6.	3.7 V Li-Ion Battery	89	4	356
7.	TP4056 Charger Module	19	1	19
8.	Slide Switch	19	2	38
9.	Battery Holder	29	1	29
10.	Jumper Wires	2	20	40
Total			₹13,312	

Table A2.3

Appendix 03

A03.1 Raspberry Pi 4 Model B:

The Raspberry Pi 4 Model B acts as the central processing unit, executing the core logic for real-time computer vision, machine learning inference, and hardware actuation.

Key Specifications:

- Processor: Broadcom BCM2711, Quad-core ARM Cortex-A72 64-bit SoC
- RAM: 4 GB (varies by model)
- Operating Voltage: 5V DC (minimum 3A required for stable operation)
- Operating System: Raspberry Pi OS
- Connectivity: Bluetooth 5.0, Wi-Fi (802.11ac), USB 3.0/2.0, dual micro-HDMI, 40-pin GPIO

Application in Project:

Processes the live video feed, runs the TensorFlow Lite MobileNet SSD model to detect humans, and outputs precise GPIO signals to the motor drivers for real-time tracking and movement.

A03.2 TB6612FNG Motor Driver:

The TB6612FNG is a compact, highly efficient dual-channel H-bridge motor driver capable of independently controlling the speed and direction of two DC motors.

Key Specifications:

- Motor Supply Voltage (VM): 15V Max (typically operates at 6V to 12V in robotics)
- Logic Supply Voltage (VCC): 2.7V to 5.5V (compatible with Raspberry Pi 3.3V logic)
- Output Current: 1.2A continuous per channel (3.2A peak)
- Control Interface: PWM (Speed) and IN1/IN2 (Direction) pins
- Features: Built-in thermal shutdown circuit and low-voltage detecting circuit

Application in Project:

Acts as the intermediary between the Raspberry Pi and the DC motors. It receives low-current GPIO control signals (logic and PWM) from the Pi and safely switches high-current power from the battery to drive the robot's wheels.

A03.3 Raspberry Pi Camera Module v1.3:

The Raspberry Pi Camera Module v1.3 is a custom-designed optical add-on that connects directly to the Pi's hardware processing pipeline, providing high-definition video capture with low latency.

Key Specifications:

- Sensor: 5 Megapixel Omnivision OV5647
- Still Resolution: 2592 x 1944 pixels
- Video Resolution: 1080p30, 720p60, and 640x480p60/90
- Interface: 15-pin MIPI Camera Serial Interface (CSI)
- Operating Voltage: 3.3V (supplied directly by the Raspberry Pi motherboard)

Application in Project:

Acts as the visual sensory input for the robot. It continuously captures real-time video frames (configured down to 320x240 for processing efficiency) and feeds them into the computer vision pipeline to scan for human targets.

A03.4 Power Supply System:

The power supply system is the energy backbone of the robot, designed to provide stable, isolated power to both the sensitive logic circuitry and the high-current electromechanical motors.

Key Specifications:

- Logic Power (Raspberry Pi): 5V DC, 3A capacity (typically via a buck converter)
- Motor Power (Drivers): High-discharge DC source (typically a 7.4V to 12V Li-ion battery pack)
- Voltage Regulation: Step-down (buck) converters to ensure voltage does not exceed component limits
- Distribution: Parallel wiring to separate motor noise from logic processing

Application in Project:

Powers the entire autonomous system. It ensures the Raspberry Pi receives an uninterrupted 5V to prevent system crashes during CPU-intensive machine learning tasks, while simultaneously delivering raw, high-current power to the TB6612FNG drivers to move the robot's physical mass.